

**UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA INFORMATICĂ ROMÂNĂ**

LUCRARE DE LICENȚĂ

**Aplicabilitatea modelelor AI de
estimare a adâncimii monocular în
detectia și evitarea obstacolelor**

Conducător științific

**Laura Silvia Dioșan
Alexandru Manole**

*Absolvent
Ștefan - Lucian Grămadă*

2026

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION INFORMATICĂ ROMÂNĂ

DIPLOMA THESIS

Applicability of monocular depth estimation AI models in obstacle detection and avoidance

Supervisor

Laura Silvia Dioșan
Alexandru Manole

Author
Ștefan - Lucian Grămadă

2026

ABSTRACT

Această lucrare prezintă proiectarea și implementarea unui sistem de asistență pentru persoanele cu deficiență de vedere, menit să faciliteze navigarea independentă prin evitarea obstacolelor, axat pe un echilibru optim între acuratețe și accesibilitate financiară. Motivația proiectului derivă din nevoia de a oferi o soluție tehnologică performantă la un cost redus, facilitând astfel accesul la scară largă. Implementarea propusă utilizează unitatea de procesare Raspberry Pi 5 (8GB) împreună cu acceleratorul hardware AI HAT+. Această configurație susține execuția unui model monocular de tip DepthAnythingV3, capabil să estimeze adâncimea dintr-un flux video captat de o singură cameră.

Cuprins

1	Introducere	1
2	Lucrări Conexe	4
2.1	Calibrarea Camerelor și Anularea Distorsiunilor	4
2.2	Modele AI pentru Estimarea Adâncimilor	5
2.3	Filtrarea Planului de Mers	7
2.4	Optimizarea Traiectoriilor	8
3	Metodologie	9
3.1	Arhitectura Generală	9
3.2	Caracteristicile Dispozitivelor Hardware Folosite	9
3.3	Calibrarea Camerei	10
3.4	Modelul Monocular de Estimare a Adâncimii	12
3.4.1	Depth Anything 3 (Platformă NVIDIA)	12
3.4.2	Depth Anything 2 ViT-S (Platformă Raspberry Pi)	13
3.5	Detecția Solului	13
3.5.1	Proiecția Inversă și Eșantionarea Punctelor Seed	14
3.5.2	Estimarea Planului prin RANSAC Vectorizat	14
3.5.3	Netezirea Temporală și Construirea Măștii	15
3.6	Detecția Drumului de Cost Minim	15
3.6.1	Determinarea Punctelor de Start și de Sfârșit	15
3.6.2	Algoritmul A* cu Conectivitate 8-Direcțională	16
4	Arhitectura Aplicației	17
4.1	Prezentare Generală	17
4.2	Tehnologii Utilizate	17
4.3	Principii de Proiectare Orientată pe Obiecte	17
4.3.1	Separarea Responsabilităților	17
4.3.2	Încapsulare și Gestiunea Stării	19
4.3.3	Configurație Tipizată prin Dataclasses	19
4.3.4	Comunicare Asincronă prin Semnale Qt	19

4.3.5	Inversiunea Dependențelor la Nivel de Backend	20
4.4	Diagrama de Clase	20
4.5	Arhitectura pe Straturi	21
4.5.1	Stratul de Prezentare (GUI)	21
4.5.2	Stratul de Orchestrare (Pipeline)	21
4.5.3	Stratul de Procesare (Core)	22
4.5.4	Stratul de Infrastructură	22
4.6	Diagrama Straturilor	23
4.7	Diagrama de Secvență	23
4.8	Gestiunea Fișierelor de Calibrare (CRUD)	24
4.9	Instrumentarea pentru Profilarea Latenței	25
4.10	Fluxul de Date	26
5	Rezultate	28
5.1	Aproximarea Adâncimii	28
5.2	Detecția Solului	30
5.3	Determinarea Drumului de Cost Minim	33
5.4	Latență	33
6	Concluzii și Direcții viitoare	36
	Bibliografie	38

Capitolul 1

Introducere

Conform raportului oficial al Organizației Mondiale a Sănătății [Wor19], aproximativ 2,2 miliarde de oameni trăiesc cu o formă de deficiență de vedere, dintre care cel puțin 1 miliard au o deficiență care ar fi putut fi prevenită sau care nu a fost încă tratată. Impactul funcțional este cel mai sever în rândul celor 43,3 milioane de persoane care suferă de orbire totală și al celor 295 de milioane cu deficiențe moderate până la severe (MSVI) [B⁺21]. Din perspectivă clinică, nevoia de asistență pentru deplasare devine universală odată ce acuitatea vizuală scade sub pragul de 3/60.

În ceea ce privește mecanismele de sprijin, deși utilizarea câinilor-ghid este considerată în literatura de reabilitare drept reperul pentru fluiditatea mișcării, accesul global rămâne extrem de scăzut. Datele furnizate de International Guide Dog Federation [Int23] relevă existența a doar aproximativ 30.000 de câini activi la nivel mondial, ceea ce înseamnă că mai puțin de 0,1% din populația care ar avea nevoie clinică de un astfel de sprijin beneficiază efectiv de el. Această disparitate este accentuată de barierele economice, costul de antrenament al unui singur exemplar fiind estimat între 40.000\$ și 60.000\$ [S⁺18]. În acest context, majoritatea persoanelor cu deficiențe de vedere rămân dependente fie de bastonul alb (utilizat de aproximativ 33,8% din populația vizată), fie de asistența umană directă [S⁺21].

O evoluție tehnologică recentă, cu un impact semnificativ asupra pieței dispozitivelor portabile (*wearable technology*), este reprezentată de ascensiunea ochelarilor inteligente (*Smart Glasses*) 1.1. Această categorie de dispozitive a fost popularizată recent prin parteneriate strategice, precum cel dintre Meta și Ray-Ban [Met], care au integrat senzori optici performanți în accesorii de uz cotidian. Din punct de vedere tehnic, prezența camerei video pe aceste dispozitive permite captarea și procesarea fluxului vizual în timp real, oferind astfel o platformă potențială pentru sisteme de asistență computațională.

Cu toate acestea, deși ochelarii Ray-Ban Meta oferă funcționalități AI impresionante, precum recunoașterea obiectelor și a scenelor din jur, traducerea în timp real, identificarea textului din mediul înconjurător și asistentul vocal integrat bazat pe

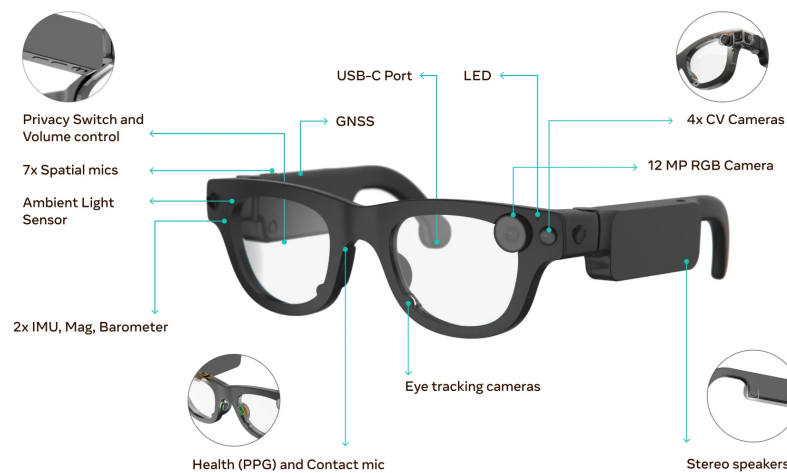


Figura 1.1: Exemplu de dispozitiv de tip Smart Glasses, Aria Gen 2

Meta AI [Met], această tehnologie puternică rămâne limitată în privința utilității pentru persoanele cu deficiențe de vedere. Cea mai utilizată aplicație dedicată acestui segment al populației este Be My Eyes [Be], o platformă care conectează utilizatorii nevăzători cu voluntari ce le oferă asistență vizuală în timp real prin intermediul camerei telefonului. Limitarea fundamentală a acestei soluții constă în dependența de disponibilitatea voluntarilor umani, ceea ce o face inadecvată pentru scenarii de navigație autonomă continuă sau pentru situații în care intervenția imediată este esențială.

În acest context considerăm că hardware-ul existent și inovațiile recente din domeniul inteligenței artificiale ar permite dezvoltarea de aplicații care să permită persoanelor cu deficiențe de vedere, o independență mult mai ridicată care nu se bazează pe ghizi canini.

Motivată de nevoia critică de independență a persoanelor cu deficiențe severe de vedere și valorificând accesibilitatea actuală a tehnologiei de tip *Smart Glasses*, prezenta lucrare propune dezvoltarea unui sistem compact și eficient din punct de vedere al costurilor. Obiectivul principal al sistemului este de a sprijini utilizatorul în detectarea și evitarea obstacolelor, facilitând astfel o mobilitate mai sigură și mai autonomă.

Lucrarea de față abordează tocmai acest gol: investigarea și implementarea unui sistem de detecție și evitare a obstacolelor destinat persoanelor cu deficiențe de vedere, construit pe o platformă accesibilă din punct de vedere financiar, în special Raspberry Pi [Rasa]. Procesul de dezvoltare a debutat pe o platformă de tip x86 echipată cu o unitate de procesare grafică dedicată, urmând ca soluția să fie ulterior portată pe platforma țintă. Această strategie de dezvoltare în două etape a condus la diferențe măsurabile de performanță între cele două configurații hardware,

diferențe datorate în principal capacității de procesare superioare oferite de GPU-ul dedicat. Din acest motiv, pe parcursul lucrării ambele platforme sunt analizate comparativ, iar rezultatele obținute pe fiecare sunt prezentate în paralel. Deși platforma țintă rămâne Raspberry Pi, rezultatele superioare înregistrate pe platforma de dezvoltare sunt incluse ca referință pentru a ilustra potențialul maxim al algoritmilor propuși.

Structura lucrării este organizată după cum urmează: Capitolul 2 trece în revistă literatura de specialitate ce a stat la baza creionării sistemului descris în lucrare. Ulterior, în Capitolul 3 este prezentată metodologia de dezvoltare și sunt analizate dificultățile tehnice întâmpinate. Capitolul 4 descrie arhitectura aplicației propuse. În Capitolul 5 sunt prezentate și discutate performanțele obținute pe cele două platforme hardware. În final, Capitolul 6 sintetizează concluziile lucrării și propune direcții viitoare de cercetare.

Capitolul 2

Lucrări Conexе

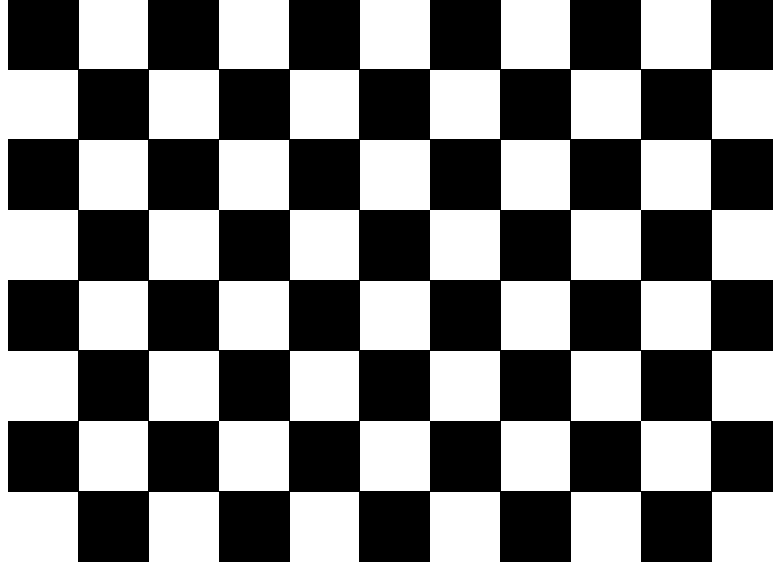
Acest capitol prezintă lucrările și metodele pe care se bazează soluția propusă. Sunt discutate, în ordine, calibrarea camerelor și corectarea distorsiunilor optice, metodele de estimare a adâncimii din imagini monoculare și multi-vizuale folosind modele bazate pe rețele neuronale, algoritmul RANSAC pentru filtrarea planului de mers din date ce conțin zgomot, precum și algoritmul A* pentru optimizarea traiectoriilor. Fiecare secțiune descrie principiile de funcționare ale metodelor relevante și modul în care acestea sunt utilizate sau adaptate în cadrul lucrării de față.

2.1 Calibrarea Camerelor și Anularea Distorsiunilor

Fundamentul oricărei analize metrice dintr-o imagine este stabilit prin calibrarea camerei. În lucrarea sa de referință [Zha00], Zhang propune o tehnică flexibilă care a democratizat accesul la viziunea 3D prin eliminarea necesității unor aparate de calibrare costisitoare. Autorul demonstrează că utilizarea unui model plan (checkerboard) 2.1 observat din cel puțin două orientări diferite este suficientă pentru a determina parametrii intrinseci și extrinseci ai camerei. Metoda se bazează pe calculul homografiei între planul modelului și proiecția sa în imagine, urmată de o rafinare neliniară prin criteriul verosimilității maxime care ia în considerare și distorsiunile radiale ale lentilei. Această abordare a transformat calibrarea dintr-un proces restrictiv de laborator într-o procedură aplicabilă în scenarii de utilizare reală.

Calibrarea estimează matricea A și coeficienții de distorsiune radială prin minimizarea erorii de reproiecție pe un set de imagini ale unui șablon de tip tablă de șah cu dimensiuni cunoscute. Soluția constă dintr-o estimare inițială în formă închisă, rafinată ulterior prin algoritmul Levenberg-Marquardt [Zha00].

Parametrii obținuți sunt utilizați în două etape ale procesării. În primul rând, fiecare cadru este rectificat geometric pentru a elimina distorsiunile radiale ale obiectivului, asigurând consistența cu modelul pinhole [HZ04]. În al doilea rând, lungi-



www.calib.io | 8x11 | Checker Size: 15 mm.

Figura 2.1: Exemplu de checkerboard pattern utilizat pentru a efectua calibrarea camerei, având 8 linii, 11 coloane (6 linii interne, 9 coloane interne) în care fiecare latură a unui pătrat măsoară 15 milimetrii pe o suprafață plană dreptunghiulară cu lungimea de 200 milimetrii, și lățimea de 150 de milimetrii, generat de platforma <https://calib.io>.

mea focală medie $f = (\alpha + \beta)/2$ este utilizată pentru conversia ieșirii modelului de adâncime în metri, conform formulei [LCL⁺25]:

$$d_{\text{metric}} = \frac{f \cdot d_{\text{raw}}}{300} \quad (2.1)$$

unde d_{raw} reprezintă valoarea brută estimată de model, iar constanta 300 provine din transformarea spațiului canonic al camerei utilizată în antrenarea modelului [LCL⁺25]. În absența calibrării, lungimea focală este estimată presupunând un câmp vizual orizontal de 70, permițând funcționarea sistemului cu o precizie metrică redusă.

2.2 Modele AI pentru Estimarea Adâncimilor

Odată stabilit modelul geometric, sarcina centrală devine inferarea adâncimii din imagini monoculare. Evoluția acestei sarcini este urmărită cel mai bine prin familia de modele *Depth Anything*, prezentată în continuare în ordine cronologică.

Punctul de plecare îl reprezintă Depth Anything V1, introdus de Yang et al. [YKH⁺24a] în cadrul conferinței Computer Vision and Pattern Recognition (CVPR) din 2024. Contribuția centrală a acestei lucrări nu constă într-un modul architectural

nou, ci în scalarea masivă a datelor de antrenament: autorii proiectează un motor de date (*data engine*) care colectează și etichetează automat aproximativ 62 de milioane de imagini neetichetate, alături de cele 1,5 milioane de imagini etichetate disponibile. Din punct de vedere arhitectural, modelul combină un encoder de tip DINOv2 [ODM⁺24] cu un decodor DPT și estimează adâncime relativă de tip afin-invariant. Pentru ca volumul mare de date să fie valorificat, sunt propuse două strategii: impunerea unui obiectiv de optimizare mai dificil prin augmentări puternice, care forțează modelul să caute activ indicii vizuale suplimentare, și o supraveghere auxiliară care transferă priorii semantici bogăți din encoderul preantrenat. Rezultatul este un model fundațional cu o capacitate de generalizare *zero-shot* remarcabilă pe imagini din medii necunoscute, putând fi rafinat ulterior pentru adâncime metrică pe seturi precum NYUv2 [SHKF12] și KITTI [GLU12].

Construind direct peste acest fundament, Yang et al. [YKH⁺24b] introduc Depth Anything V2, care schimbă sursa principală a supervizării: în locul imaginilor reale etichetate manual, antrenarea se bazează pe date sintetice, ale căror etichete de adâncime sunt absolut precise și permit învățarea detaliilor geometrice fine. Pentru a depăși decalajul de domeniu dintre imaginile sintetice și cele reale, V2 păstrează ideea scalării datelor din V1, dar o reorganizează într-o schemă de tip profesor-elev (*teacher-student*): un model profesor de mare capacitate, antrenat exclusiv pe date sintetice, generează etichete pseudo-adâncime pe un volum masiv de imagini reale neetichetate, pe care apoi sunt antrenate modelele-elev. Față de V1, această modificare aduce hărți de adâncime mai robuste și cu o granularitate net superioară, în special în zonele cu structuri fine și obiecte transparente, păstrând în același timp o eficiență computațională ridicată.

Extinzând aceste capacități către o înțelegere multi-vizuală, Depth Anything 3 (DA3) [LCL⁺25] propune o metodă de recuperare a spațiului vizual din orice număr de intrări, indiferent dacă poziția camerei este cunoscută sau nu. Contribuția majoră a DA3 constă în demonstrarea faptului că o arhitectură simplă, bazată pe un transformator standard (vanilla DINO), este suficientă pentru a obține rezultate de ultimă oră fără a fi nevoie de specializări arhitecturale complexe. Prin utilizarea unui target de predicție de tip „depth-ray”, modelul reușește o consistență spațială remarcabilă, depășind standardele anterioare în acuratețea geometrică și în estimarea poziției camerei cu peste 23% față de predecesorii săi.

Pentru a sintetiza evoluția celor trei generații, Tabelul 2.1 compară dimensiunea modelelor, iar Tabelul 2.2 rezumă performanța în estimarea adâncimii relative (zero-shot) și metrice. La nivel de capacitate, se observă că abia V2 introduce varianta uriașă (*Giant*, ~1,3 miliarde de parametri), folosită ca model profesor, în timp ce DA3 menține aceeași gamă de dimensiuni, dar cu un backbone ușor mai compact. În privința metricilor clasice de adâncime, valorile sunt aproape identice între V1 și

V2: progresul real al V2 se manifestă în detaliile fine și robustețe, aspecte care, așa cum notează chiar autorii [YKH⁺24b], nu sunt surprinse de aceste benchmark-uri standard. DA3 [LCL⁺25] aduce câștiguri vizibile mai ales pe seturi precum ETH3D și își concentrează evaluarea pe acuratețea geometrică și pe estimarea poziției camerei, motiv pentru care nu raportează rezultate metrice monoculare în același format ca predecesorii.

Variantă (encoder)	V1	V2	DA3
Small (ViT-S)	24,8M	24,8M	22M
Base (ViT-B)	97,5M	97,5M	86M
Large (ViT-L)	335,3M	335,3M	300M
Giant (ViT-g)	—	1,3B	1,13B

Tabela 2.1: Numărul de parametri pentru variantele celor trei generații *Depth Anything* [YKH⁺24a, YKH⁺24b, LCL⁺25]. *Notă:* pentru V1 și V2 cifrele provin din depozitele oficiale și includ modelul complet (encoder DINOv2 + decodor DPT), în timp ce pentru DA3 valorile raportate corespund în principal backbone-ului DINOv2; numerele nu sunt, prin urmare, măsurate identic.

Set de date	V1		V2		DA3
	AbsRel↓	δ_1 ↑	AbsRel↓	δ_1 ↑	δ_1 ↑
<i>Estimare relativă (zero-shot)</i>					
KITTI	0,076	0,947	0,074	0,946	0,953
NYU	0,043	0,981	0,045	0,979	0,974
Sintel	0,458	0,760	0,487	0,752	0,755
ETH3D	0,127	0,882	0,131	0,865	0,986
DIODE	0,066	0,952	0,066	0,952	0,954
<i>Estimare metrică (fine-tuned)</i>					
NYUv2	0,056	0,984	0,056	0,984	—
KITTI	0,046	0,982	0,045	0,983	—

Tabela 2.2: Performanța în estimarea adâncimii (encoder ViT-L), AbsRel (↓, mai mic e mai bun) și δ_1 (↑, mai mare e mai bun). Rezultatele V1/V2 provin din [YKH⁺24a, YKH⁺24b], iar coloana δ_1 pentru DA3 din [LCL⁺25] (Tabel 4, reevaluare monoculară). *Notă:* DA3 raportează doar δ_1 pentru adâncimea relativă și nu publică rezultate metrice monoculare în acest format, de unde marcajul „—”.

2.3 Filtrarea Planului de Mers

În procesarea datelor extrase din mediul vizual, prezența zgomotului și a erorilor sistematice este inevitabilă. Fischler și Bolles [FB81] au introdus algoritmul RANSAC (Random Sample Consensus) ca o soluție pentru potrivirea modelelor mate-

matice pe seturi de date ce conțin un procent ridicat de erori brute (outliers). Spre deosebire de tehnicile de tip „least squares”, RANSAC utilizează un proces iterativ de selecție a unui subset minim de date pentru a genera un model candidat, verificând apoi consensul acestuia cu restul datelor. Această paradigmă este vitală în analiza imaginilor, unde detectorii de trăsături pot genera corespondențe eronate; RANSAC permite izolarea modelului corect (cum ar fi un plan sau o poziție a camerei) prin ignorarea zgomotului care nu se conformează structurii geometrice globale.

2.4 Optimizarea Traietoriilor

În final, având o reprezentare stabilă a spațiului și a obstacolelor, problema deplasării este redusă la găsirea unui drum optim. Hart et al. [HNR68] au oferit baza formală pentru determinarea căilor de cost minim prin algoritmul A*. Acesta utilizează o funcție euristică pentru a ghida procesul de căutare într-un graf, evaluând fiecare nod prin suma dintre costul parcurs de la start (g) și costul estimat până la țintă (h). Autorii demonstrează că dacă euristica h este admisibilă (adică nu supraestimează niciodată costul real), algoritmul A* garantează găsirea căii optime. Această cercetare a stabilit echilibrul matematic între eficiența căutării și certitudinea soluției, rămânând o piatră de temelie pentru orice sistem de navigație care operează în spații de stare complexe.

Capitolul 3

Metodologie

3.1 Arhitectura Generală

Sistemul utilizează o cameră video pentru achiziția de date vizuale în timp real. Deoarece obiectivele camerelor introduc frecvent distorsiuni geometrice (în special spre extremitățile cadrului), este necesară o etapă preliminară de calibrare pentru rectificarea imaginilor. Parametrii de calibrare sunt stocați în sistem, astfel încât procesul nu necesită rulări repetate, deși utilizatorul poate opta pentru o nouă calibrare oricând este necesar.

După captare și rectificare, cadrul video este procesat de un model de estimare monoculară a adâncimii pentru a genera o hartă de adâncime (*depth map*). Următoarea etapă constă în identificarea suprafeței de rulare. În această lucrare, algoritmul de detecție vizează exclusiv planuri orizontale, netede și continue, pe care le vom defini generic sub termenul de „sol”.

În final, integrând datele din harta de adâncime cu geometria planului solului, sistemul stabilește două puncte de referință în spațiul 3D proiectat. Un algoritm de planificare a traseului (*pathfinding*) calculează apoi drumul de lungime minimă între aceste puncte, evitând obstacolele detectate în scenă. Fluxul logic al procesării datelor este sintetizat în diagrama 3.1.

3.2 Caracteristicile Dispozitivelor Hardware Folosite

Dezvoltarea sistemului s-a desfășurat în două etape distincte. Într-o primă fază, implementarea și testarea au fost realizate pe un laptop echipat cu placă grafică dedicată, urmând ca ulterior sistemul să fie adaptat și optimizat pentru a rula pe un dispozitiv cu resurse limitate, respectiv un Raspberry Pi în combinație cu un accelerator neuronal dedicat.

Laptopul utilizat este un Lenovo Legion Pro 5 16IAX10H, echipat cu un procesor

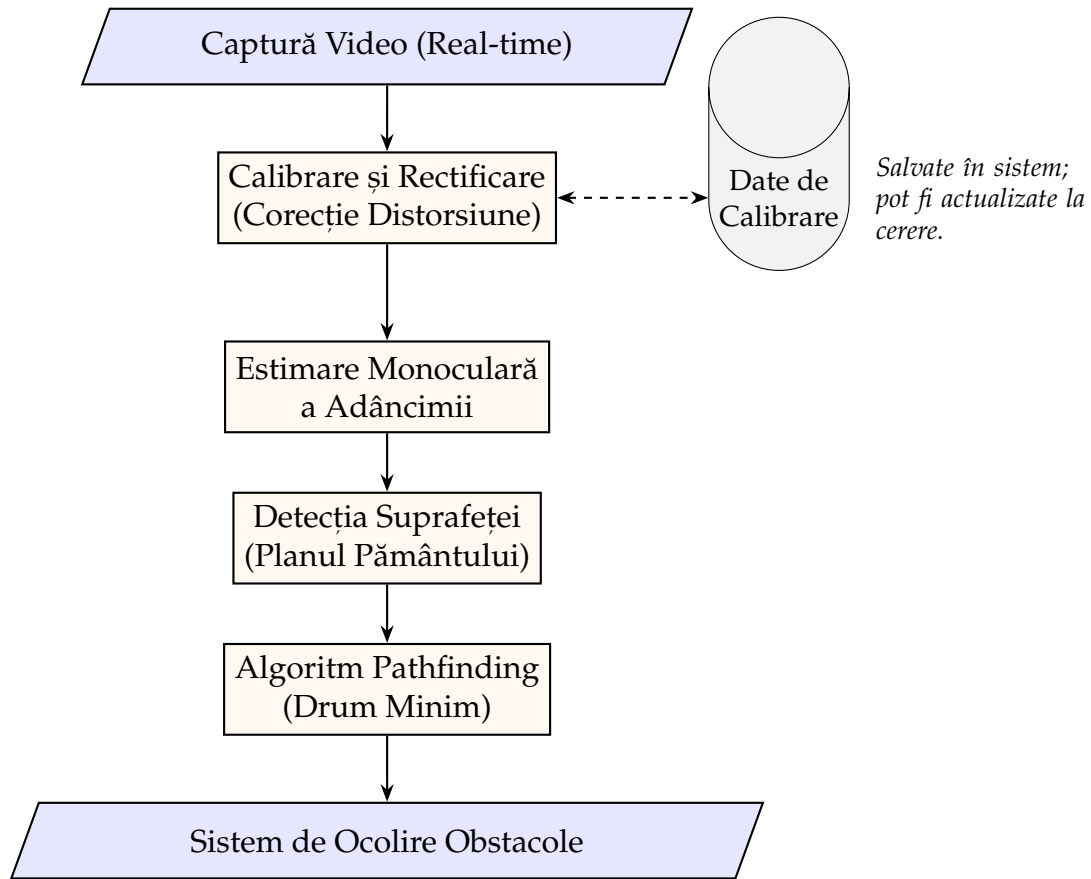


Figura 3.1: Diagrama fluxului de date și a procesării în sistemul de asistență la mobilitate.

Intel Core Ultra 9 275HX, cu 24 de nuclee și o frecvență maximă de 5,40 GHz [Int], și o placă grafică NVIDIA GeForce RTX 5070 Ti Mobile, cu 12 GB memorie VRAM dedicată și un TDP configurabil de până la 140W.

Platforma embedded este reprezentată de un Raspberry Pi 5 cu 8 GB RAM [Rasa], la care s-a atașat modulul Raspberry Pi AI HAT+, un accelerator neuronal cu o putere de procesare de 26 TOPS, bazat pe procesorul Hailo-8 [Rasb]. Implicațiile acestei configurații asupra performanței sistemului, atât avantajele, cât și limitările impuse, sunt discutate în detaliu în secțiunea 3.4, dedicată modelelor monoculare de estimare a adâncimii.

3.3 Calibrarea Camerei

Calibrarea camerei reprezintă un pas esențial în obținerea unor estimări metrice precise ale adâncimii, în special atunci când se utilizează un model monocular metric de estimare a adâncimii. Procesul de calibrare determină parametrii intrinseci ai camerei, distanța focală și punctul principal, precum și coeficienții de distorsiune, care modelează deformările geometrice introduse de lentilă, cel mai frecvent manifestate

la marginile imaginii. Fără corectarea acestor distorsiuni, estimările de adâncime ar acumula erori ce pot afecta semnificativ precizia sistemului de evitare a obstacolelor.

Metoda utilizată se bazează pe tehnica de calibrare propusă de Zhang [Zha00], implementată în biblioteca OpenCV. Aceasta presupune captarea mai multor imagini ale unui panou de calibrare cu model de tablă de șah (*checkerboard*) din unghiuri și distanțe variate, permițând estimarea robustă a parametrilor camerei prin minimizarea erorii de reproiecție.

Algoritmul de calibrare utilizează patru parametri configurabili: `cols`, `rows`, `square_mm` și `min_frames`. Parametrii `cols` și `rows` definesc numărul de colțuri interioare detectabile ale tablei de șah pe orizontală, respectiv verticală. Parametrul `square_mm` specifică dimensiunea reală a laturii unui pătrat, exprimată în milimetri, valoare necesară pentru a stabili corespondența dintre coordonatele din spațiul real și cele din planul imaginii. În fine, `min_frames` indică numărul minim de cadre ce trebuie captate pentru ca estimarea parametrilor să fie suficient de stabilă. Toți acești parametri pot fi ajustați din secțiunea *Settings* a aplicației, oferind flexibilitate în funcție de camera utilizată.

Din punct de vedere tehnic, pentru fiecare cadru în care panoul de calibrare este detectat, algoritmul rafinează pozițiile colțurilor detectate la nivel subpixel folosind metoda `cornerSubPix` din OpenCV, îmbunătățind precizia corespondenței puncte-imagine. Odată acumulate suficiente cadre (`min_frames`), se apelează `cv.calibrateCamera` care returnează matricea intrinsecă a camerei K și vectorul coeficienților de distorsiune d .

Calitatea calibrării este evaluată prin eroarea RMS (*Root Mean Square*) de reproiecție, care măsoară discrepanța medie, în pixeli, între colțurile observate și cele reproiectate pe baza parametrilor estimați. O valoare RMS sub 1.0 este considerată acceptabilă pentru aplicații de tip vedere artificială [Zha00]. În implementarea curentă, dacă valoarea depășește acest prag, utilizatorul este avertizat și recalibrarea este recomandată.

Pentru a evita repetarea procesului de calibrare la fiecare pornire a aplicației, rezultatele sunt persistate într-un fișier `.npz` (format NumPy arhivat), care stochează matricea camerei, coeficienții de distorsiune și valoarea RMS. Numele implicit al fișierului este `calibration.npz`, însă acesta poate fi modificat din secțiunea *Settings*. De asemenea, utilizatorul poate exporta fișierul de calibrare sau poate importa unul existent, cu condiția că acesta a fost generat utilizând aceiași parametri intrinseci și extrinseci.

3.4 Modelul Monocular de Estimare a Adâncimii

Estimarea adâncimii dintr-o singură imagine (estimare monoculară) reprezintă o problemă fundamentală în vederea artificială, cu aplicații directe în navigație autonomă și asistarea persoanelor cu deficiențe de vedere. Spre deosebire de metodele stereoscopice sau LiDAR, abordarea monoculară nu necesită hardware specializat suplimentar, ceea ce o face potrivită pentru sisteme portabile cu resurse limitate.

În cadrul acestui sistem au fost utilizate două modele distincte, selectate în funcție de platforma hardware disponibilă: **Depth Anything 3** pentru platforma NVIDIA și **Depth Anything 2 ViT-S** pentru Raspberry Pi 5 cu acceleratorul AI HAT+.

3.4.1 Depth Anything 3 (Platformă NVIDIA)

Depth Anything 3 [LCL⁺25] reprezintă cea mai recentă iterație a familiei de modele Depth Anything, concepută pentru a depăși limitările versiunilor anterioare în ceea ce privește consistența spațială și generalizarea la scene diverse. În cadrul acestui sistem este utilizată varianta DA3METRIC-LARGE, care produce estimări de adâncime în unități metrice absolute, eliminând necesitatea unei etape suplimentare de scalare relativă.

Modelul este încărcat prin intermediul bibliotecii PyTorch și transferat pe GPU-ul dedicat al laptopului, beneficiind de accelerarea CUDA. Pentru maximizarea performanței, inferența utilizează precizie mixtă automată (*Automatic Mixed Precision* — AMP), care reduce consumul de memorie VRAM și crește debitul de procesare fără a afecta semnificativ precizia rezultatelor. De asemenea, la inițializare se încearcă compilarea statică a modelului prin `torch.compile()`, disponibilă începând cu PyTorch 2.0, care poate aduce îmbunătățiri suplimentare de latență pe hardware compatibil.

Conversia ieșirii brute a rețelei la adâncime metrică se realizează conform formulei oficiale din documentația modelului:

$$d_{\text{metric}} = \frac{f \cdot r}{300} \quad (3.1)$$

unde r reprezintă predicția brută a rețelei (valoare adimensională), iar f este distanța focală medie a camerei, calculată din matricea intrinsecă $\mathbf{K}$ drept $f = (f_x + f_y)/2$. În absența unei calibrări disponibile, distanța focală este estimată pornind de la o presupunere de câmp vizual orizontal de aproximativ 70, rezultând valori aproximative ale adâncimii metrice.

3.4.2 Depth Anything 2 ViT-S (Platformă Raspberry Pi)

Pentru platforma embedded, utilizarea Depth Anything 3 nu este fezabilă din cauza cerințelor computaționale ridicate ale acestuia și acceleratorului neuronal ce necesită recompilarea modelului special pentru platformă. În schimb, este utilizat modelul **Depth Anything 2** [YKH⁺24b] în varianta **ViT-S** (*Vision Transformer Small*), compilat în formatul HEF (*Hailo Executable Format*) specific procesorului Hailo-8 din cadrul modulului AI HAT+. Această compilare este furnizată prin Hailo Model Zoo și permite rularea eficientă a modelului pe NPU-ul dedicat, descărcând complet procesorul ARM al Raspberry Pi-ului de sarcina de inferență.

Pipeline-ul de inferență pe această platformă cuprinde trei etape distincte. În etapa de pre-procesare, cadrul RGB de intrare este redimensionat la rezoluția așteptată de model (518×518 pixeli) și convertit la formatul `uint8 NHWC`, normalizarea fiind gestionată intern de HEF. Inferența propriu-zisă este realizată prin SDK-ul oficial `hailort`, utilizând *virtual streams* pentru comunicarea cu NPU-ul prin interfața PCIe. În etapa de post-procesare, modelul DA2 ViT-S produce o hartă de disparitate inversă (adâncime relativă inversă), în care valorile mai mari corespund obiectelor mai apropiate. Pentru a asigura consistența cu restul pipeline-ului aplicației, unde valorile mai mari de adâncime corespund distanțelor mai mari, harta este inversată și normalizată la intervalul $[0, 1]$ conform relației:

$$d_{\text{norm}} = \frac{d_{\text{max}} - d}{d_{\text{max}} - d_{\text{min}}} \quad (3.2)$$

Această abordare implică o limitare importantă: adâncimile produse pe Raspberry Pi sunt *relative*, nu metrice, ceea ce înseamnă că sistemul nu poate determina distanțele absolute față de obstacole pe această platformă. În consecință, logica de evitare a obstacolelor bazată pe algoritmul A* [HNR68] operează în acest caz cu valori normalizate, iar pragurile de detecție trebuie ajustate corespunzător față de configurația NVIDIA.

3.5 Detecția Solului

Detecția suprafeței solului reprezintă o etapă preliminară esențială pentru algoritmul de evitare a obstacolelor, întrucât permite excluderea zonei practicabile din harta de adâncime înainte de a identifica obstacolele relevante. Fără această etapă, solul însuși ar fi interpretat ca un obstacol, perturbând planificarea traseului. Abordarea adoptată se bazează pe algoritmul RANSAC (*Random Sample Consensus*) [FB81], aplicat în spațiul 3D reconstituit din harta de adâncime și parametrii intrinseci ai camerei.

3.5.1 Proiecția Inversă și Eșantionarea Punctelor Seed

Prima etapă constă în selecția unui subset de puncte candidate pentru estimarea planului solului. Deoarece solul este vizibil cu precădere în zona inferioară a imaginii, algoritmul eșantionează doar o fracțiune configurabilă din partea de jos a hărții de adâncime (parametrul `seed_region`). Pentru a limita costul computațional, se aplică un pas de subșantionare (`stride = 4`), reducând numărul de pixeli evaluați, iar dacă numărul de puncte valide depășește un prag maxim (`max_points = 1500`), se aplică o selecție aleatoare suplimentară.

Fiecare pixel eșantionat este reproiectat din spațiul imaginii în spațiul 3D al camerei folosind relațiile geometrice standard derivate din modelul pinhole al camerei [HZ04]:

$$X = \frac{(u - c_x) \cdot z}{f_x}, \quad Y = \frac{(v - c_y) \cdot z}{f_y}, \quad Z = z \quad (3.3)$$

unde (u, v) sunt coordonatele pixelului, z este adâncimea estimată, iar f_x, f_y, c_x, c_y sunt parametrii intrinseci ai camerei din matricea $\mathbf{K}$.

3.5.2 Estimarea Planului prin RANSAC Vectorizat

Estimarea planului solului se realizează printr-o variantă vectorizată a algoritmului RANSAC [FB81], în care toate iterațiile sunt evaluate în paralel prin operații NumPy, eliminând bucla explicită și reducând semnificativ latența. La fiecare iterație, se selectează aleator câte un triplet de puncte, se calculează normala planului prin produsul vectorial, iar numărul de inlieri (puncte situate la o distanță mai mică decât pragul `threshold` față de plan) este determinat simultan pentru toate iterațiile printr-o operație matricială:

$$\text{dist}(p, \pi) = |\mathbf{n}^T p + d| \quad (3.4)$$

unde $\mathbf{n}$ este vectorul normal al planului și d este termenul liber. Planul cu cel mai mare număr de inlieri este reținut, iar parametrii săi sunt rafinați ulterior printr-o descompunere SVD pe mulțimea completă de inlieri, obținând o estimare mai robustă.

Un plan estimat este acceptat doar dacă componenta verticală a normalei sale depășește un prag configurabil (`normal_threshold`), condiție care asigură că planul detectat este suficient de orizontal pentru a fi considerat sol și nu o suprafață verticală (perete, ușă etc.).

3.5.3 Netezirea Temporală și Construirea Măștii

Pentru a evita fluctuațiile bruște între cadre consecutive, planul estimat la fiecare cadru este combinat cu planul anterior printr-o medie ponderată exponențială:

$$\pi_t = \alpha \cdot \pi_{t-1} + (1 - \alpha) \cdot \pi_{\text{raw}} \quad (3.5)$$

unde α este factorul de netezire (`plane_smoothing`), cu valori apropiate de 1 favorizând stabilitatea temporală în detrimentul reactivității la schimbări rapide ale scenei. Planul rezultat este renormalizat după fiecare actualizare pentru a menține validitatea geometrică a reprezentării.

În final, masca binară a solului este construită direct în spațiul 2D al imaginii, fără a materializa întregul nor de puncte 3D. Pentru fiecare pixel (u, v) , distanța față de planul estimat este calculată vectorizat, iar pixelii a căror distanță este sub pragul `threshold` sunt marcați ca aparținând solului. Pragul în sine este adaptat în funcție de modul de operare al modelului de adâncime: în modul metric se folosește o valoare absolută în metri (`ransac_threshold_metric`), iar în modul relativ pragul este scalat proporțional cu intervalul dinamic al hărții de adâncime curente (`ransac_threshold_relative`).

3.6 Detecția Drumului de Cost Minim

Odată determinată masca binară a solului practicabil, sistemul calculează un drum de cost minim între poziția curentă a utilizatorului și un punct țintă aflat în fața acestuia. Această problemă este formulată ca o căutare de drum pe un graf implicit definit de masca solului, rezolvată prin algoritmul A* [HNR68], cunoscut pentru optimalitatea și eficiența sa în spații de căutare cu euristici admisibile.

3.6.1 Determinarea Punctelor de Start și de Sfârșit

Punctul de start este identificat în zona inferioară a măștii solului, în apropierea centrului orizontal al imaginii, simulând poziția picioarelor utilizatorului în cadrul video. Algoritmul scanează rândurile de jos în sus și, pentru fiecare rând, caută cel mai apropiat pixel de sol față de coloana centrală, extinzând progresiv căutarea spre margini. Punctul de sfârșit este determinat simetric, prin scanare de sus în jos, reprezentând cel mai îndepărtat punct practicabil vizibil în direcția de mers.

Această convenție — start în partea de jos, țintă în partea de sus a imaginii — reflectă direct geometria perspectivei unei camere montate frontal: pixelii din partea inferioară a imaginii corespund suprafețelor apropiate, iar cei din partea superioară corespund zonelor mai îndepărtate.

3.6.2 Algoritmul A* cu Conectivitate 8-Direcțională

Graful de căutare este definit implicit de masca solului: fiecare pixel marcat ca sol reprezintă un nod, iar muchiile conectează fiecare pixel cu cei opt vecini ai săi (conectivitate 8-direcțională). Costul unui pas orizontal sau vertical este 1,0, iar costul unui pas diagonal este $\sqrt{2} \approx 1,4142$, reflectând distanța euclidiană reală dintre pixeli adiacenți.

Funcția euristică utilizată este distanța octilică (*octile distance*), care constituie o euristică admisibilă pentru conectivitatea 8-direcțională:

$$h(n, g) = \max(\Delta r, \Delta c) + (\sqrt{2} - 1) \cdot \min(\Delta r, \Delta c) \quad (3.6)$$

unde $\Delta r = |r_n - r_g|$ și $\Delta c = |c_n - c_g|$ sunt diferențele de rând și coloană dintre nodul curent n și nodul țintă g . Admisibilitatea euristicii garantează că A* returnează întotdeauna drumul de cost minim [HNR68].

Algoritmul menține o coadă de priorități (*min-heap*) ordonată după scorul $f = g + h$, unde g reprezintă costul acumulat de la start, și o matrice g_score de dimensiunea imaginii pentru stocarea celui mai bun cost cunoscut pentru fiecare nod. La finalizare, drumul este reconstruit prin urmărirea înapoi a structurii `came_from` de la nodul țintă până la cel de start.

Dacă nu există nicio cale continuă de pixeli de sol între punctul de start și cel de sfârșit — situație posibilă când un obstacol blochează complet trecerea — algoritmul returnează un drum vid, iar sistemul poate semnaliza utilizatorul în consecință.

Capitolul 4

Arhitectura Aplicației

4.1 Prezentare Generală

Aplicația este structurată ca un sistem modular de procesare video în timp real, cu scopul de a detecta planul solului și de a planifica o traiectorie navigabilă pe baza unei hărți de adâncime generate de un model de învățare profundă. Sistemul suportă două profiluri hardware distincte: un host cu GPU NVIDIA, folosind PyTorch și CUDA pentru inferență, respectiv un Raspberry Pi 5 cu accelerator NPU Hailo-8, folosind biblioteca `hailort`. Indiferent de profilul activ, restul pipeline-ului rămâne identic, ceea ce face posibilă portabilitatea codului fără modificări.

Întreaga bază de cod este scrisă în Python 3.11+ și organizată în două mari componente: un **backend** de procesare și o **interfață grafică** (GUI) construită cu PyQt6. Separarea clară dintre aceste două componente permite înlocuirea sau testarea fiecăreia independent.

4.2 Tehnologii Utilizate

Implementarea sistemului se bazează pe un set de biblioteci și framework-uri specializate, selectate în funcție de cerințele de performanță și de compatibilitatea cu cele două platforme hardware vizate. Tabelul 4.1 prezintă o sinteză a acestora, împreună cu versiunea utilizată și rolul fiecăreia în arhitectura aplicației.

4.3 Principii de Proiectare Orientată pe Obiecte

4.3.1 Separarea Responsabilităților

Fiecare modul al aplicației are o singură responsabilitate bine definită. Modulul `config.py` se ocupă exclusiv de citirea și validarea configurației. Modulul `hardware.py`

Tehnologie	Versiune	Rol în aplicație
Python	3.11+	Limbajul principal; utilizat pentru întreaga bază de cod, beneficiind de <code>dataclasses</code> , <code>type hints</code> și <code>tomllib</code> nativ
PyQt6	6.x	Interfața grafică multi-platformă; mecanismul de semnale și sloturi asigură comunicarea thread-safe între fire de execuție
OpenCV	4.x	Captură video (V4L2), corecție de distorsiune, calibrare cu tablă de șah
NumPy	2.x	Procesare numerică vectorizată a hărților de adâncime, măștilor și coordonatelor 3D
PyTorch + CUDA	2.x	Backend de inferență pe GPU NVIDIA; suportă AMP și <code>torch.compile</code>
Hailo Runtime	4.x	Backend de inferență pe NPU Hailo-8 integrat în Raspberry Pi AI HAT+
TOML	—	Format de configurație structurată, citit nativ via <code>tomllib</code> (Python 3.11+)

Tabela 4.1: Tehnologiile principale utilizate în implementare

abstractizează diferențele dintre platformele hardware. Modulul `ground.py` conține exclusiv logica de detectare a planului solului, fără nicio referință la interfața grafică sau la camera video. Această separare face ca fiecare componentă să poată fi testată, înlocuită sau extinsă fără a afecta restul sistemului.

4.3.2 Încapsulare și Gestiunea Stării

Clasele sunt folosite acolo unde există stare internă care trebuie gestionată pe durata vieții unui obiect. `HardwareProfile` încapsulează profilul hardware activ și expune proprietăți derivate (`is_nvidia`, `is_rpi`, `is_metric`) fără a expune câmpurile interne. `CalibrationService` grupează operațiile de calibrare împreună cu configurația de care au nevoie. `DepthPipeline` gestionează întregul ciclu de viață al procesării unui cadru video: deschiderea camerei, încărcarea modelului, firul de captură, și eliberarea resurselor la oprire.

Funcțiile pure sunt preferate claselor atunci când nu există stare internă de gestionat. Modulele `ground.py`, `path.py` și `visualization.py` sunt compuse exclusiv din funcții, deoarece transformările pe care le realizează (proiecție 3D, RAN-SAC, A^* , suprapunere vizuală) sunt fără efecte secundare și nu necesită obiecte.

4.3.3 Configurație Tipizată prin Dataclasses

Configurația aplicației este citită o singură dată la pornire din fișierul `config.toml` și transformată într-un arbore de `dataclass`-uri imutabile (`frozen=True`): `AppConfig`, `CameraConfig`, `CalibrationConfig`, `GroundConfig`, `ProfilingConfig` etc. Această abordare elimină accesul prin chei de tip șir de caractere și înlocuiește șirul de caractere cu atribute tipizate, ceea ce permite detectarea erorilor de configurare la încărcare, nu la rulare, și oferă autocompletare în mediile de dezvoltare.

4.3.4 Comunicare Asincronă prin Semnale Qt

Inferența modelului de adâncime și captura video rulează în fire de execuție separate față de firul principal al interfeței grafice. Comunicarea dintre fire se realizează exclusiv prin mecanismul de semnale și sloturi al Qt (`pyqtSignal`), care garantează că actualizările interfeței grafice au loc întotdeauna pe firul principal. Această abordare previne blocarea interfeței în timpul procesării și elimină condițiile de cursă (*race conditions*) specifice accesului concurent la widget-uri.

4.3.5 Inversiunea Dependențelor la Nivel de Backend

Modulul `model.py` acționează ca un registru de factory: în funcție de profilul hardware activ, deleghează apelurile fie către `model_nvidia.py`, fie către `model_rpi.py`. Pipeline-ul central (`pipeline.py`) depinde exclusiv de interfața abstractă a lui `ModelRegistry`, nu de implementările concrete. Adăugarea unui nou backend necesită doar adăugarea unui fișier nou și a unui caz în registru, fără a modifica pipeline-ul sau GUI-ul.

4.4 Diagrama de Clase

Figura 4.1 prezintă clasele principale ale aplicației și relațiile dintre acestea. Conform notației UML, săgețile continue reprezintă *asocieri*: clasa sursă păstrează o referință de durată (un câmp) către clasa țintă. Săgețile întrerupte reprezintă *dependențe*: clasa sursă doar creează sau folosește tranzitoriu un obiect, fără a-l reține ca stare. Relația de instanțiere este marcată suplimentar cu stereotipul standard `<<create>>`.

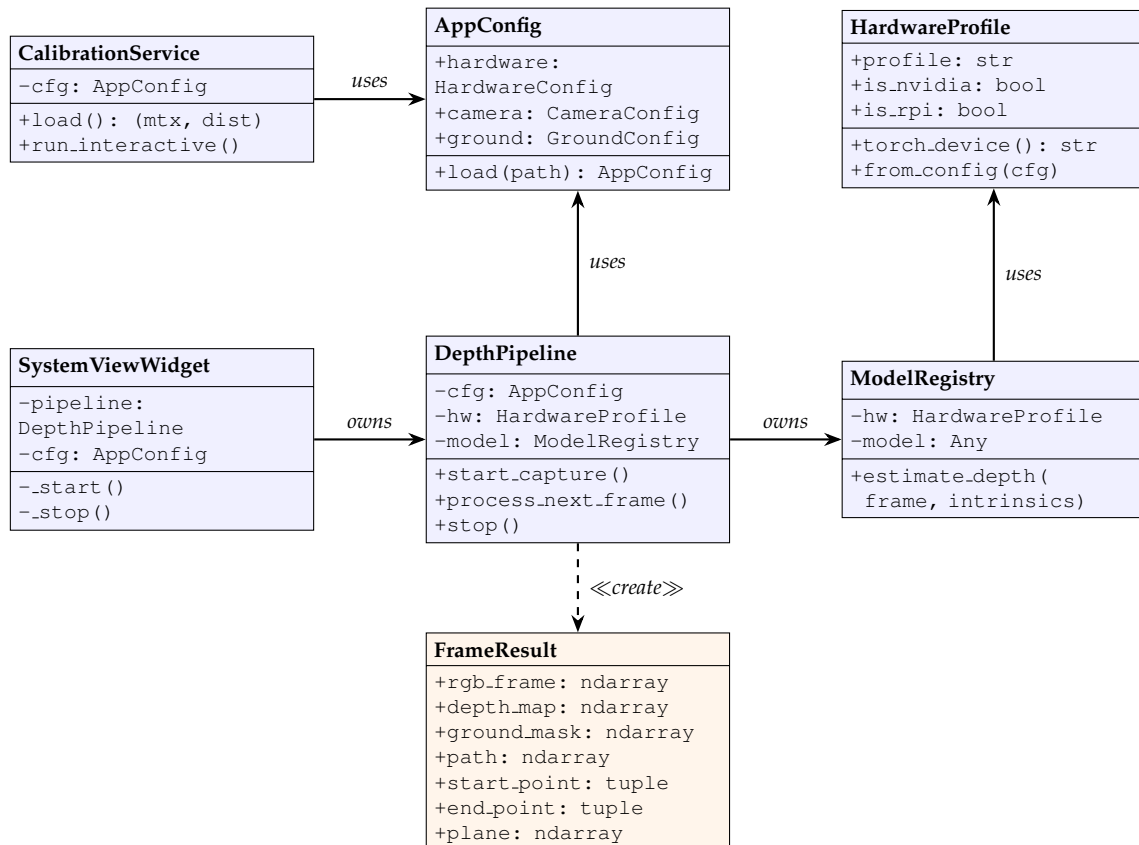


Figura 4.1: Diagrama de clase UML a componentelor principale ale aplicației.

4.5 Arhitectura pe Straturi

Aplicația este organizată în patru straturi distincte, cu dependențe strict direcționate de sus în jos. Stratul superior depinde de cel imediat inferior; niciun strat nu cunoaște existența straturilor superioare.

4.5.1 Stratul de Prezentare (GUI)

Correspunde pachetului `gui/` și este responsabil exclusiv de interacțiunea cu utilizatorul. Conține:

- `app.py` — fereastra principală (`MainWindow`), care gestionează navigarea între ecrane printr-un `QStackedWidget`;
- `main_menu.py` — ecranul de pornire, care expune acțiunile principale ale aplicației: calibrarea camerei, biblioteca de calibrări, pornirea sistemului și editarea setărilor;
- `calibration.py` — ecranul de calibrare, inclusiv dialogul modal cu vizualizare live și afișarea scorului RMS;
- `calibration_manager.py` — ecranul bibliotecii de calibrări, care expune vizual operațiile CRUD asupra calibrărilor salvate (listare, aplicare, import, export, editare a metadatelor și ștergere);
- `settings.py` — formularul de editare a configurației, cu scriere înapoi în `config.toml`;
- `system_view.py` — vizualizarea live a sistemului, cu bara laterală de opțiuni și redarea cadrelor procesate;
- `components.py` — widget-uri reutilizabile: `StyledButton`, `NavBar`, `ToggleSwitch`, `SectionHeader`, `StatusBadge`;
- `styles.py` — paleta de culori, fonturile și foaia de stiluri globală Qt.

Stratul de prezentare nu realizează nicio procesare de imagine sau inferență. El pornește și oprește pipeline-ul prin metode publice și primește rezultate prin semnale Qt.

4.5.2 Stratul de Orchestrare (Pipeline)

Correspunde modulului `pipeline.py` și reprezintă punctul central de coordonare al aplicației. Clasa `DepthPipeline` gestionează:

- un fir de captură dedicat, care citește continuu cadre de la cameră și păstrează întotdeauna cel mai recent cadru disponibil;
- un fir de inferență, care consumă cadrele, rulează pipeline-ul complet și emite rezultate prin semnale Qt;
- starea internă a sistemului: hărțile de corecție a distorsiunii, matricea intrinsecă activă, planul anterior detectat;
- un profiler de latență (`FrameProfiler`), care măsoară opțional durata fiecărei etape, detaliat în Secțiunea 4.9.

Rezultatul unui cadru procesat este reprezentat de dataclass-ul `FrameResult`, care agregă cadrul RGB, harta de adâncime, masca solului, traiectoria A*, punctele de start/stop și coeficienții planului.

4.5.3 Stratul de Procesare (Core)

Conține implementările algoritmice independente de hardware și de interfața grafică:

- `ground.py` — detectarea planului solului prin RANSAC vectorizat cu eșantionare subpixelică, urmat de rafinare SVD și construirea măștii direct în spațiul 2D al imaginii;
- `path.py` — planificarea traiectoriei prin algoritmul A* cu conectivitate pe 8 direcții și euristică octilă admisibilă;
- `visualization.py` — suprapunerea vizuală a măștii solului, a traiectoriei și a barei de stare pe cadrul video;
- `camera.py` — deschiderea camerei (V4L2 sau picamera2), construirea hărților de corecție a distorsiunii și aplicarea lor;
- `calibration.py` — captarea pozelor de calibrare și rezolvarea matricei intrinseci prin `cv2.calibrateCamera`.

4.5.4 Stratul de Infrastructură

Conține modulele care nu realizează procesare propriu-zisă, ci furnizează configurație, abstractizare hardware și acces la modele:

- `config.py` — citirea fișierului `config.toml` și construirea arborelui de dataclass-uri tipizate;

- `hardware.py` — `HardwareProfile`: abstractizarea profilului activ și selecția dispozitivului de calcul;
- `model.py` — `ModelRegistry`: factory care deleghează încărcarea și inferența modelului de adâncime;
- `model_nvidia.py` — backend PyTorch + CUDA, folosind `DepthAnything3` cu AMP și `torch.compile` opțional;
- `model_rpi.py` — backend Hailo, folosind `hailort` pentru inferența pe NPU-ul Hailo-8;
- `profiling.py` — `FrameProfiler`: instrumentarea pentru măsurarea latenței per etapă a pipeline-ului (Secțiunea 4.9);
- `main.py` — punctul de intrare în aplicație; lansează GUI-ul sau modul headless.

4.6 Diagrama Straturilor

Figura 4.2 ilustrează sintetic cei patru straturi ai arhitecturii și direcția unică a dependențelor, de la interfața grafică până la infrastructură.

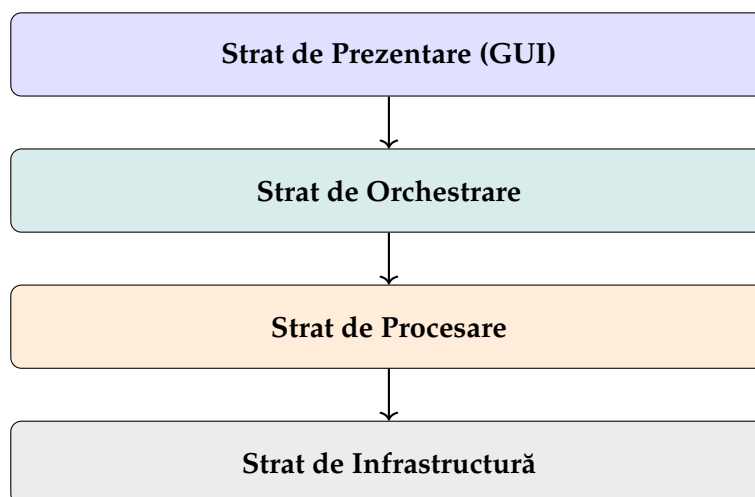


Figura 4.2: Arhitectura pe straturi a aplicației. Dependențele sunt direcționate exclusiv de sus în jos.

4.7 Diagrama de Secvență

Figura 4.3 ilustrează interacțiunea dintre componente pentru scenariul central al aplicației: pornirea sistemului de către utilizator și procesarea continuă a cadrelor

video până la oprire.

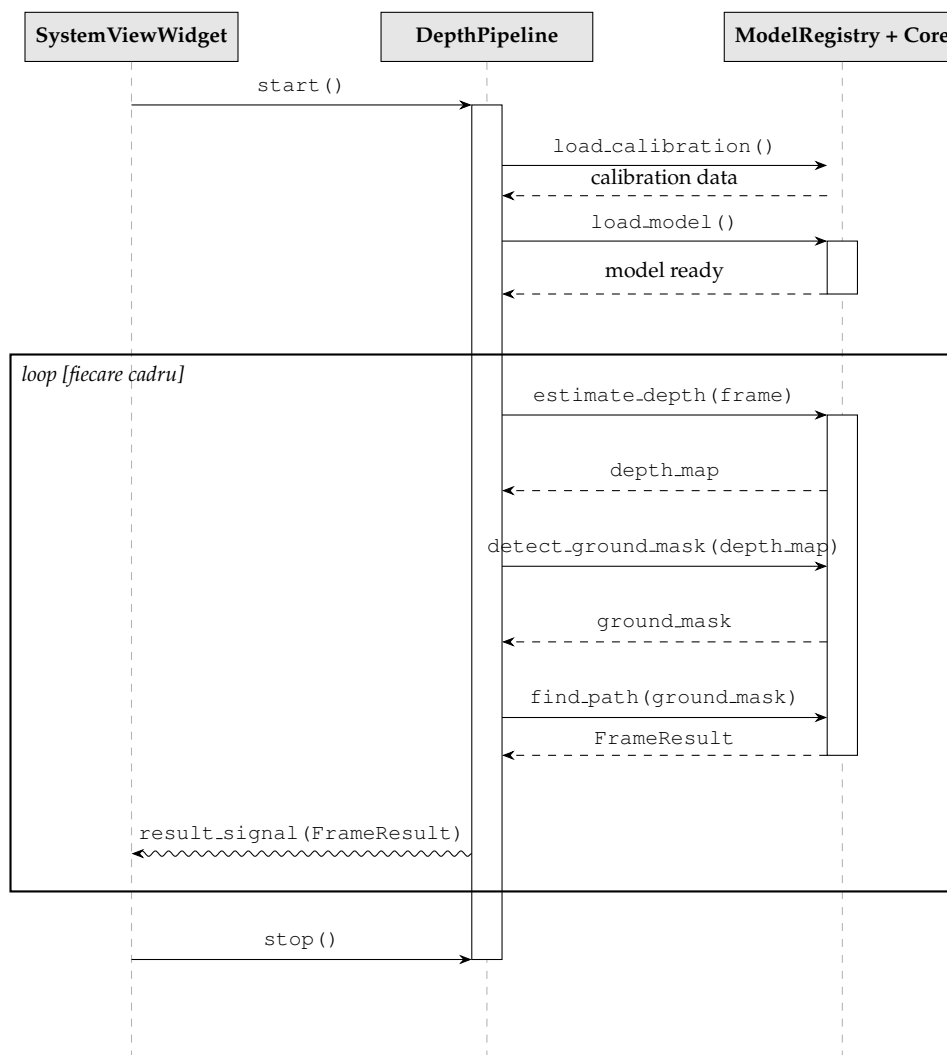


Figura 4.3: Diagrama de secvență pentru inițializare și procesarea continuă a cadrelor video. Săgeata ondulată reprezintă semnalul Qt asincron emis către GUI.

4.8 Gestiunea Fișierelor de Calibrare (CRUD)

Aplicația include un subsistem dedicat stocării și gestionării calibrărilor produse în urma sesiunilor de calibrare. Arhitectura acestuia urmează modelul clasic pe trei niveluri: domeniu, repository și serviciu.

Modelul de date. Entitatea centrală este `CalibrationRecord`, un dataclass pur Python fără dependențe ORM sau NumPy, care conține metadatele unei calibrări: scorul RMS, dimensiunile grilei (`cols`, `rows`), dimensiunea pătratului în milimetri, un nume și note opționale, data creării și calea către fișierul `.npz` asociat. Persistența se realizează printr-o bază de date SQLite (`calibrations.db`), gestionată prin SQLAlchemy, iar fișierele `.npz` sunt stocate fizic în directorul `calibration_files/`

sub un nume UUID unic generat la salvare.

Nivelul de acces la date. Clasa `CalibrationRepository` implementează operațiile de bază asupra bazei de date: `save()` pentru inserarea unui record nou, `find_by_id()` și `find_all()` pentru interogare, `update()` pentru modificarea metadatelor și `delete()` pentru ștergerea unui rând. Conversia între obiectul de domeniu și modelul ORM este realizată prin metodele statice `to_model()` și `to_domain()`, menținând astfel o separare strictă între straturile de persistență și logica aplicației.

Nivelul de serviciu. `CalibrationLibraryService` expune operațiile CRUD complete cu validare integrată:

- **Create** — `add_from_calibration()` copiază automat fișierul `.npz` produs de o sesiune de calibrare în directorul gestionat și salvează metadatele în baza de date; `import_file()` permite importul unui fișier extern ales de utilizator, extragând RMS-ul și parametrii grilei direct din fișier dacă sunt disponibili;
- **Read** — `list_all()` returnează toate calibrările ordonate descrescător după dată; `apply()` copiază fișierul `.npz` al unei calibrări selectate în locația așteptată de pipeline (`calibration.npz`), activând-o; `export()` permite salvarea fișierului într-o destinație aleasă de utilizator;
- **Update** — `update_metadata()` permite editarea numelui și a notelor asociate unei înregistrări, cu validare de lungime și conținut;
- **Delete** — `delete()` șterge atât înregistrarea din baza de date, cât și fișierul `.npz` de pe disc, asigurând consistența între cele două spații de stocare.

Validarea este delegată clasei `CalibrationValidator`, care verifică RMS-ul, integritatea fișierului `.npz` și lungimea câmpurilor text înainte de orice operație de scriere.

4.9 Instrumentarea pentru Profilarea Latenței

Pentru evaluarea cantitativă a performanței (prezentată în Capitolul 5), aplicația include un subsistem de instrumentare care măsoară durata fiecărei etape a pipeline-ului, cadru cu cadru. Proiectarea acestuia urmărește două cerințe aparent contradictorii: să ofere măsurători precise atunci când profilarea este activă, dar să nu introducă niciun cost atunci când este dezactivată, pentru a nu denatura comportamentul normal al sistemului.

Strategie interschimbabilă. Pipeline-ul comunică întotdeauna cu o interfață abstractă, `FrameProfiler`, fără a cunoaște implementarea concretă. Există două strategii (*Strategy pattern*): `NullProfiler`, folosit când profilarea este dezactivată,

ale cărui metode sunt operații vide (*Null Object*) și care returnează un singur context reutilizabil, astfel încât calea critică nu plătește practic nimic; și `LatencyProfiler`, care cronometrează efectiv etapele. Funcția-fabrică `make_profiler()` construiește strategia potrivită pe baza configurației, eliminând astfel ramificațiile condiționale din interiorul buclei de procesare.

Măsurare per etapă. Fiecare etapă sincronă a pipeline-ului — corecția distorsiunii, estimarea adâncimii, detecția solului și determinarea drumului — este încadrată într-un context `manager_profiler.stage(name)`, care măsoară durata blocului cu `time.perf_counter()`. Suma acestor patru etape constituie timpul total de procesare per cadru. Captura cadrului, care rulează pe firul de captură separat, este raportată informativ, dar exclusă din total. La finalul fiecărui cadru, `commit_frame()` agregă timpii acumulați într-un singur eșantion.

Scriere în loturi. Pentru a evita operații de intrare/ieșire pe calea critică, eșantioanele sunt acumulate în memorie și scrise pe disc în loturi (implicit la fiecare 50 de cadre și la oprire). Fișierul CSV rezultat este denumit automat astfel încât să codifice profilul hardware și modul de adâncime sub care au fost colectate datele (de exemplu `latency_nvidia_metric_20260604_153012.csv`), păstrând măsurătorile legate de condițiile în care au fost obținute. Orice eroare de scriere este izolată, astfel încât profilarea să nu poată întrerupe niciodată pipeline-ul.

Activare în timpul rulării. Comutatorul *Profile latency* din bara laterală a ecranului de vizualizare permite pornirea și oprirea profilării fără a reporni sistemul. Metoda `set_profiling_enabled()` înlocuiește referința strategiei active — pornirea inițiază un nou fișier CSV, iar oprirea îl golește și îl închide. Deoarece înlocuirea referinței este atomică sub GIL, operația este sigură față de firul de inferență care citește strategia activă.

4.10 Fluxul de Date

La pornirea sistemului, `MainWindow` încarcă configurația tipizată din `config.toml` printr-un apel la `AppConfig.load()`. Când utilizatorul apasă butonul *START* din ecranul de vizualizare live, `SystemViewWidget` instanțiază un `DepthPipeline` și pornește firul de inferență. Acesta inițializează `ModelRegistry` (care, în funcție de profilul hardware, încarcă fie modelul PyTorch, fie fișierul HEF pentru Hailo), deschide camera prin `CameraFactory` și pornește firul de captură.

La fiecare cadru disponibil, pipeline-ul aplică corecția de distorsiune, rulează inferența pentru a obține harta de adâncime, detectează planul solului prin RANSAC vectorizat și construiește masca, planifică traiectoria prin A*, și împachetează toate rezultatele într-un obiect `FrameResult`. Dacă profilarea este activă, durata fiecăreia dintre aceste etape este măsurată și înregistrată (Secțiunea 4.9). Acesta este

transmis stratului de prezentare prin semnalul Qt `result_signal`, unde `SystemViewWidget` aplică suprapunerile vizuale și actualizează widget-ul de afișare.

Utilizatorul poate modifica parametrii sistemului în orice moment prin ecranul *Settings*, care scrie valorile actualizate înapoi în `config.toml` și semnalizează ferestrei principale să reîncarce configurația. Modificările intră în vigoare la următoarea pornire a pipeline-ului.

Capitolul 5

Rezultate

În acest capitol sunt prezentate rezultate obținute în condiții reale de utilizare. Sunt analizate atât cazuri în care fiecare componentă a sistemului funcționează conform așteptărilor, cât și situații în care o etapă a pipeline-ului produce rezultate inexacte, pentru a evidenția deopotrivă capabilitățile și limitările sistemului.

Testarea sistemului bazat pe Raspberry Pi necesită un monitor, dispozitive periferice (mouse și tastatură) și o sursă de alimentare capabilă să livreze 5,1 V la 5 A, o configurație disponibilă rar pe piață. Din acest motiv, colectarea de date pe această platformă a fost limitată, iar pentru evaluarea vizuală s-a utilizat un singur scenariu, comparat cu sistemul bazat pe platforma NVidia. În fiecare secțiune este analizată mai întâi platforma NVidia, urmată de comparația cu Raspberry Pi.

Secțiunile 5.1, 5.2 și 5.3 se bazează pe o evaluare vizuală, folosind imagini capturate în timpul rulării. Secțiunea 5.4 prezintă o evaluare cantitativă a latenței.

5.1 Aproximarea Adâncimii

Hărțile de adâncime prezentate în această secțiune au fost vizualizate prin aplicarea unei palete de culori de tip *inferno*: punctele apropiate de cameră sunt reprezentate în tonuri închise, iar cele îndepărtate în tonuri deschise.

Pe platforma bazată pe NVidia, estimarea adâncimii produce rezultate consistente în scenariile diferite, atât interioare, cât și exterioare. Figurile 5.3 și 5.6 prezintă două astfel de exemple.

Platforma bazată pe Raspberry Pi dispune de resurse de calcul reduse și utilizează un model dintr-o generație anterioară, cu un număr mai mic de parametri. În consecință, estimarea adâncimii este mai puțin detaliată, diferența fiind vizibilă în special la nivelul muchiilor. Comparația dintre harta de adâncime și imaginea originală din figura 5.9 permite identificarea elementelor pe care modelul reușește totuși să le distingă.



Figura 5.1: Imagine exterioră captată după aplicarea corecției.

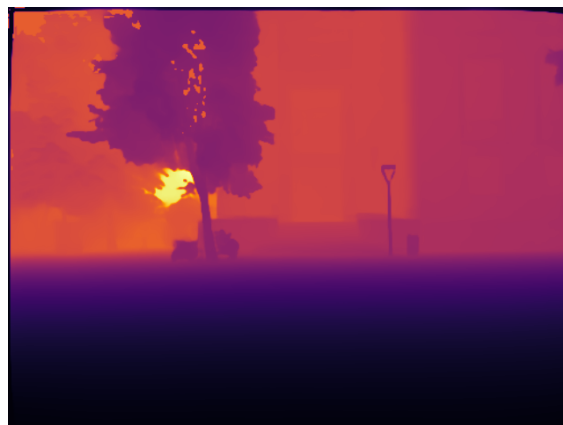


Figura 5.2: Harta de adâncime estimată.

Figura 5.3: Comparație între imaginea originală și harta de adâncime într-un scenariu exterior.

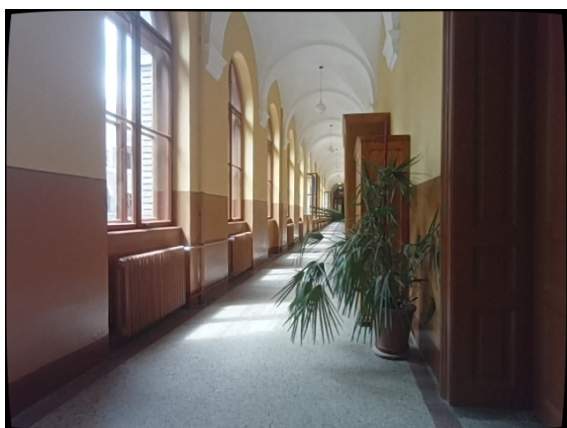


Figura 5.4: Imagine interioară captată după aplicarea corecției.

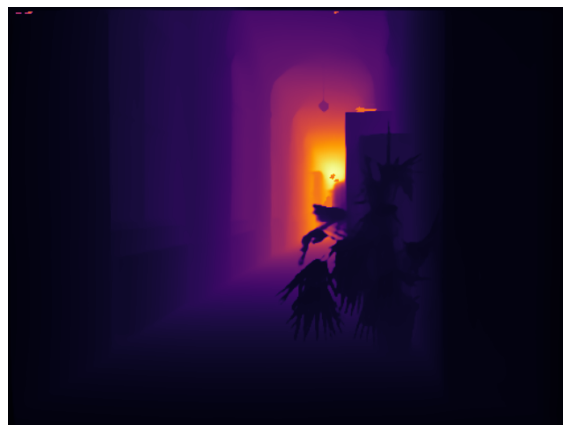


Figura 5.5: Harta de adâncime estimată.

Figura 5.6: Comparație între imaginea originală și harta de adâncime într-un scenariu interior.



Figura 5.7: Imagine interioară captată după aplicarea corecției (Raspberry Pi + Hailo-8).

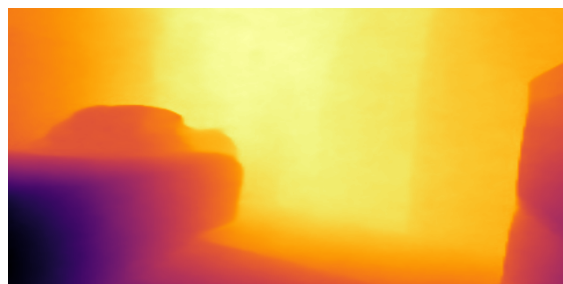


Figura 5.8: Harta de adâncime estimată (Raspberry Pi + Hailo-8).

Figura 5.9: Comparație între imaginea originală și harta de adâncime într-un scenariu interior (Raspberry Pi + Hailo-8).

5.2 Detectia Solului

În majoritatea cazurilor, detectia solului identifică corect planul de mers, așa cum se observă în figura 5.10.

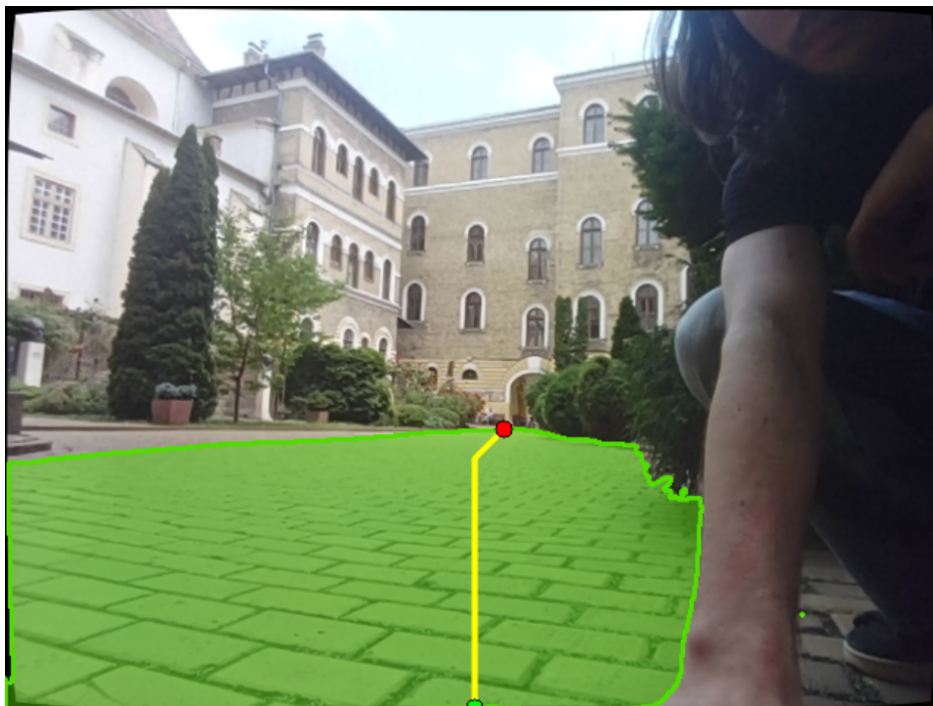


Figura 5.10: Exemplu de detecție corectă a planului solului.

În cazul traseelor lungi, planul detectat nu cuprinde întreaga suprafață: punctele aflate la distanță mare nu sunt incluse în planul solului (figura 5.13). Acest comportament este dificil de explicat, întrucât harta de adâncime corespunzătoare nu prezintă, vizual, anomalii evidente. Identificarea cauzei necesită investigații suplimentare.

O limitare a metodei de detecție a solului decurge din principiul său de funcționare: metoda identifică planul în care se află majoritatea punctelor din harta de adâncime. În cele mai multe situații acest plan coincide cu planul solului, însă în spații înguste ipoteza nu mai este valabilă, iar planul este identificat eronat, așa cum se observă în figura 5.14.

De asemenea, în anumite situații planul de mers detectat conține porțiuni neacoperite (figura 5.15). Acest efect poate fi atribuit hărții de adâncime.

Exemplele prezentate până în acest punct provin de la sistemul bazat pe NVidia. Comparatia cu platforma Raspberry Pi indică influența pe care precizia hărții de adâncime o are asupra detecției solului: pe Raspberry Pi planul solului nu este identificat corect, așa cum se observă în figura 5.16.

O altă limitare este faptul că metoda nu distinge între un trotuar și o stradă cu trafic auto. Această abordare a fost aleasă pentru viteza de procesare mai mare com-

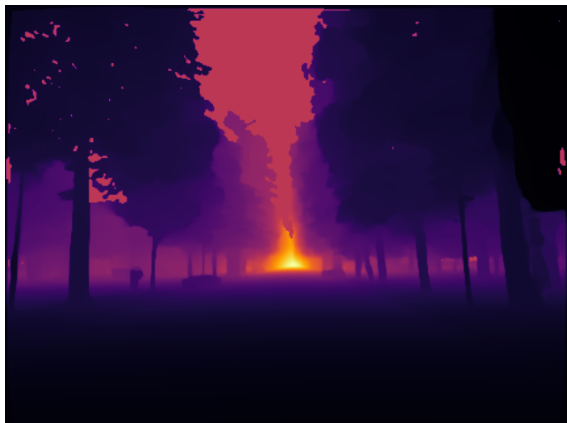


Figura 5.11: Harta de adâncime corespunzătoare.



Figura 5.12: Planul solului detectat, incomplet la distanță mare.

Figura 5.13: Detecția solului pe un traseu lung: harta de adâncime nu prezintă anomalii vizibile, însă planul detectat nu include punctele îndepărtate.

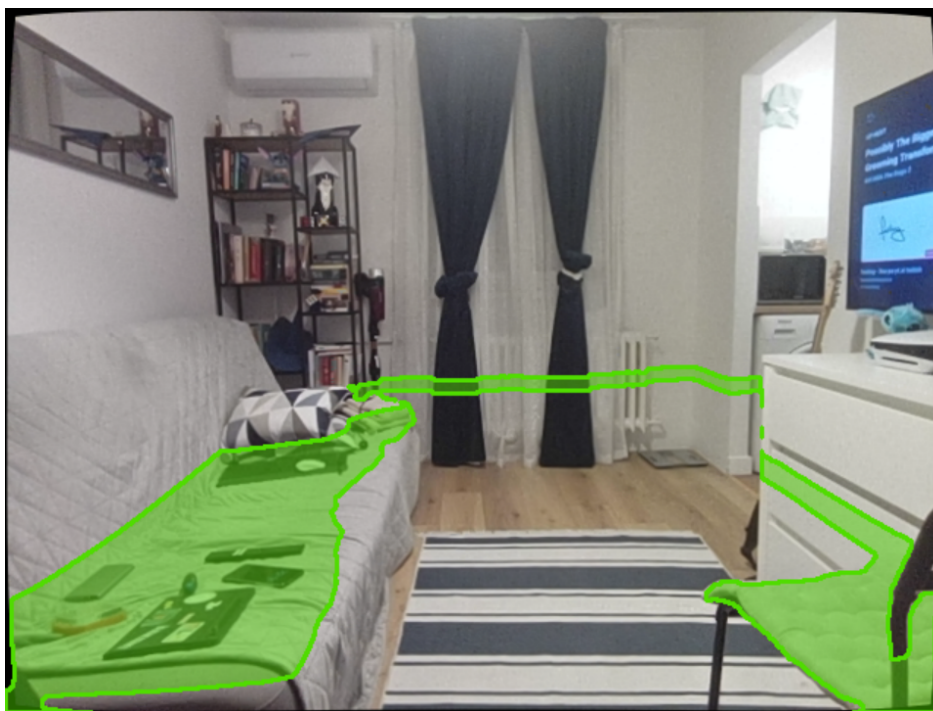


Figura 5.14: Identificare eronată a planului solului într-un spațiu îngust.



Figura 5.15: Plan al solului detectat cu porțiuni neacoperite.



Figura 5.16: Exemplu de detecție incorectă a planului solului pe platforma Raspberry Pi.

parativ cu metodele bazate pe rețele neuronale, în concordanță cu obiectivul de a construi un sistem compact, cu buget redus și constrângeri hardware.

5.3 Determinarea Drumului de Cost Minim

Drumul de cost minim nu este calculat pe proiecția tridimensională a planului de mers, ci pe proiecția acestuia în plan bidimensional. Această abordare nu garantează identificarea drumului de cost minim din spațiul real în toate situațiile, însă în majoritatea cazurilor rezultatul este suficient pentru scopul propus. Figura 5.17 prezintă un exemplu de drum determinat, suprapus peste harta de adâncime și planul solului detectat.

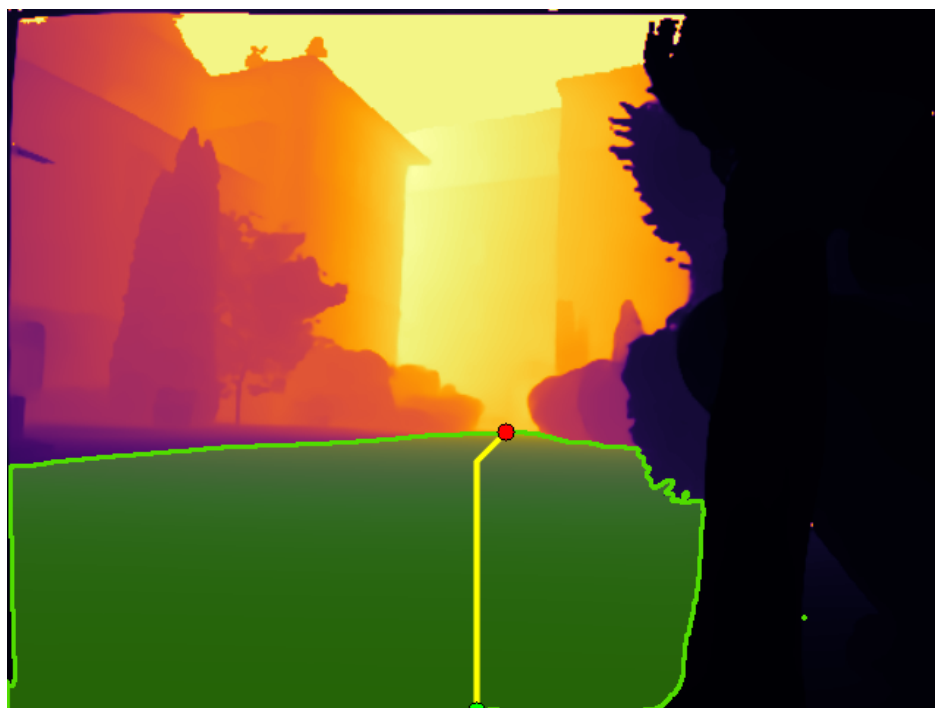


Figura 5.17: Exemplu de drum de cost minim determinat, suprapus peste harta de adâncime și planul solului detectat.

5.4 Latență

Spre deosebire de secțiunile anterioare, evaluarea latenței se bazează pe date cantitative colectate de modulul de profilare integrat în aplicație. Acesta măsoară, cu ajutorul funcției `time.perf_counter()`, durata fiecărei etape a pipeline-ului pentru fiecare cadru procesat și o înregistrează într-un fișier CSV, în loturi, pentru a nu introduce întârzieri suplimentare pe calea critică. Statisticile prezentate în continuare au fost obținute pe platforma bazată pe NVidia (rezoluție 640×480 , model

Depth Anything 3 metric, varianta Large), agregând aproximativ 9.989 de cadre din 13 sesiuni de rulare distincte. Primul cadru al fiecărei sesiuni a fost exclus, întrucât include inițializarea modelului și nu reflectă regimul normal de funcționare.

Pipeline-ul este compus din patru etape sincrone — corecția distorsiunii, estimarea adâncimii, detecția solului și determinarea drumului — a căror sumă constituie timpul total de procesare per cadru. Capturarea cadrului de la cameră se desfășoară pe un thread separat, în paralel cu procesarea, motiv pentru care nu este inclusă în timpul total, deși este raportată informativ. Rezultatele agregate sunt prezentate în tabelul 5.1.

Etapă	Medie [ms]	Mediană [ms]	Dev. std. [ms]	p95 [ms]
Capturare cadru*	34,1	32,1	4,0	42,4
Corecția distorsiunii	0,5	0,5	1,7	0,7
Estimarea adâncimii	64,1	64,1	9,1	69,5
Detecția solului	0,9	0,9	0,3	1,1
Determinarea drumului	47,7	13,7	200,9	212,3
Total procesare	113,2	79,6	200,9	276,6

Tabela 5.1: Statistici de latență per etapă pe platforma NVidia, agregate peste 9.989 de cadre din 13 sesiuni de rulare (cadrul de inițializare exclus). *Capturarea cadrului se execută pe un thread separat și nu este inclusă în timpul total de procesare.

Estimarea adâncimii reprezintă componenta dominantă a timpului de procesare, cu aproximativ 56% din total (64,1 ms în medie). Aceasta este singura etapă bazată pe o rețea neuronală și constituie principala limitare a cadenței, având totodată o variabilitate redusă (deviație standard de 9,1 ms). Etapele pur geometrice — corecția distorsiunii și detecția solului — au un cost redus, sub 1 ms fiecare, ceea ce este consistent cu decizia de a folosi o metodă bazată pe RANSAC pentru detecția solului, în locul unei a doua rețele neuronale.

Etapă de determinare a drumului are un cost median redus (13,7 ms), însă distribuția sa prezintă valori extreme: aproximativ 19% dintre cadre depășesc 50 ms, iar în cazuri nefavorabile căutarea drumului poate dura câteva secunde. Aceasta explică diferența dintre medie (47,7 ms) și mediană și se reflectă în cei doi indicatori ai cadenței. Pornind de la mediana timpului total (79,6 ms), sistemul atinge aproximativ 12,6 cadre pe secundă, în timp ce media (113,2 ms) corespunde unei valori de aproximativ 8,8 cadre pe secundă. Diferența dintre cele două valori este determinată de cadrele cu cost ridicat în etapa de determinare a drumului.

Pe platforma NVidia, latența este determinată în principal de modelul de estimare a adâncimii, celelalte componente având un cost redus și predictibil. O optimizare a latenței ar viza, în primul rând, accelerarea inferenței modelului de adâncime

și, în al doilea rând, reducerea variabilității etapei de determinare a drumului.

Aceleași măsurători au fost realizate și pe sistemul bazat pe Raspberry Pi + Hailo-8, pentru a evalua influența platformei hardware asupra latenței. Spre deosebire de platforma NVidia, estimarea adâncimii este delegată acceleratorului neuronal Hailo-8 și utilizează un model de dimensiuni reduse — Depth Anything 2, varianta ViT-S, la rezoluția 224×224 , în mod relativ. Statisticile, agregate peste aproximativ 2.086 de cadre din 2 sesiuni de rulare, sunt prezentate în tabelul 5.2.

Etapă	Medie [ms]	Mediană [ms]	Dev. std. [ms]	p95 [ms]
Capturare cadru*	59,1	53,9	32,2	122,8
Corecția distorsiunii	3,8	3,3	5,3	7,1
Estimarea adâncimii	42,3	41,8	1,8	45,7
Detecția solului	6,1	5,6	2,2	10,4
Determinarea drumului	83,4	36,7	105,4	226,3
Total procesare	135,5	90,8	104,8	277,7

Tabela 5.2: Statistici de latență per etapă pe platforma Raspberry Pi + Hailo-8, agregate peste 2.086 de cadre din 2 sesiuni de rulare (cadrul de inițializare exclus).

*Capturarea cadrului se execută pe un thread separat și nu este inclusă în timpul total de procesare.

Rezultatele indică un profil de latență diferit față de platforma NVidia. Pe Raspberry Pi, estimarea adâncimii are un cost redus (42,3 ms în medie, cu o deviație standard de 1,8 ms), ca urmare a modelului de dimensiuni reduse executat pe acceleratorul neuronal. În schimb, componenta dominantă devine etapa de determinare a drumului (83,4 ms în medie, aproximativ 61% din total), executată pe procesorul ARM. Și în acest caz distribuția prezintă valori extreme, peste 43% dintre cadre depășind 50 ms. Latențele estimării adâncimii nu sunt însă direct comparabile între cele două platforme, întrucât modelele utilizate diferă ca generație și dimensiune, așa cum s-a discutat în secțiunea 5.1.

În ansamblu, cele două platforme ating cadențe apropiate — aproximativ 11,0 cadre pe secundă (mediană) pe Raspberry Pi, față de 12,6 pe NVidia — dar limitate de componente diferite: sistemul NVidia este limitat de inferența modelului de adâncime, în timp ce sistemul Raspberry Pi, deși execută inferența pe acceleratorul neuronal, este limitat de calculele geometrice efectuate pe procesorul ARM. În consecință, pe platforma embedded, optimizarea ar viza în primul rând etapa de determinare a drumului.

Capitolul 6

Concluzii și Direcții viitoare

Lucrarea de față a propus și a implementat un sistem de detecție și evitare a obstacolelor destinat persoanelor cu deficiențe de vedere, construit pe un pipeline compus din corecția distorsiunii, estimarea monoculară a adâncimii, detecția planului solu-lui și planificarea drumului. Sistemul a fost dezvoltat și evaluat pe două platforme hardware cu capacități diferite: o platformă bazată pe GPU NVIDIA, utilizată pen-tru dezvoltare, și platforma țintă, un Raspberry Pi 5 cuplat cu acceleratorul neuronal Hailo-8. Rezultatele obținute pe cele două configurații conturează atât capabilitățile sistemului, cât și limitările care orientează direcțiile de cercetare ulterioare.

În condiții normale de iluminare, pe platforma NVIDIA, modelul Depth Any-thing 3 în varianta metric Large produce estimări de adâncime consistente în sce-narii interioare și exterioare (secțiunea 5.1), suficiente pentru ca etapele ulterioare ale pipeline-ului să funcționeze conform așteptărilor. Pe platforma țintă situația diferă: din rațiuni de resurse de calcul și de compatibilitate cu acceleratorul neuro-nal, estimarea adâncimii este realizată de modelul Depth Anything 2 ViT-S, dintr-o generație anterioară și cu un număr redus de parametri. Adâncimea rezultată este mai puțin detaliată (figura 5.9), iar această limitare se propagă în aval: detecția solu-lui nu mai identifică în mod corect planul de mers (figura 5.16). Estimarea adâncimii constituie, prin urmare, componenta care determină calitatea rezultatelor pe plat-forma embedded.

O limitare comună ambelor platforme decurge din natura senzorului utilizat. O cameră video convențională este dependentă de nivelul de lumină pe care îl primește, iar estimarea monoculară a adâncimii, fiind o metodă pasivă, moștenește această dependență. În condiții de iluminare redusă, calitatea cadrului captat scade, ceea ce afectează acuratețea hărții de adâncime și, implicit, a întregului pipeline.

Detecția solului reprezintă o a doua limitare. Metoda adoptată, bazată pe algo-rismul RANSAC, identifică planul în care se află majoritatea punctelor din harta de adâncime și îl asimilează planului de mers. Această ipoteză este simplă din punct de vedere geometric și nu se verifică în toate situațiile: în spații înguste planul este

identificat eronat (figura 5.14), iar pe traseele lungi suprafața detectată rămâne incompletă (secțiunea 5.2). În plus, metoda nu distinge între un trotuar și o stradă cu trafic auto, întrucât operează exclusiv pe baza geometriei planului, nu a conținutului semantic al scenei. O abordare bazată pe segmentare semantică prin rețele neuronale ar putea adresa aceste situații, cu prețul unui cost computațional mai ridicat; varianta geometrică a fost aleasă pentru viteza de procesare și pentru încadrarea în constrângerile de buget și de resurse ale platformei țintă.

O direcție de dezvoltare ulterioară vizează valorificarea capacității multi-vizuale a modelului Depth Anything 3 (secțiunea 2.2). Spre deosebire de estimarea dintr-un singur cadru, reconstrucția spațiului tridimensional din mai multe cadre, captate din unghiuri diferite, ar permite menținerea informației acumulate în cadrele anterioare. O astfel de consistență temporală ar putea îmbunătăți atât detecția solului, cât și determinarea drumului, prin completarea zonelor ocluzionate sau parțial vizibile la un moment dat cu informație observată anterior.

O direcție alternativă constă în înlocuirea estimării monoculare a adâncimii cu un senzor LIDAR. Un astfel de senzor furnizează măsurători metrice directe, independente de condițiile de iluminare, și ar elimina necesitatea unui model de estimare a adâncimii, cu un câștig de acuratețe și de predictibilitate a performanței. Acest avantaj este însă condiționat de o complexitate hardware mai mare și de un cost ridicat, aspecte aflate în tensiune cu obiectivul declarat al lucrării, acela de a construi un sistem compact și accesibil din punct de vedere financiar (secțiunea 3.4).

În ansamblu, sistemul propus demonstrează fezabilitatea unei soluții de asistență la mobilitate construite pe componente accesibile, evidențiind totodată că factorul determinant al performanței rămâne calitatea estimării adâncimii, condiționată la rândul ei de platforma hardware și de senzorul utilizat. Direcțiile prezentate — segmentarea semantică a solului, reconstrucția multi-vizuală a scenei și utilizarea unui senzor activ de tip LIDAR — conturează un compromis recurent între acuratețe, cost și resurse de calcul, care va ghida evoluția ulterioară a sistemului.

Bibliografie

- [B⁺21] Rupert R. A. Bourne et al. Causes of blindness and vision impairment in 2020 and trends over 30 years, and prevalence of avoidable blindness in 2020: a systematic review and meta-analysis. *The Lancet Global Health*, 9(2):e144–e160, 2021.
- [Be] Be My Eyes. Be my eyes – making the world more accessible. <https://www.bemyeyes.com/>. Accesat: mai 2025.
- [FB81] Martin A. Fischler and Robert C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM*, 24(6):381–395, June 1981.
- [GLU12] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3354–3361, 2012.
- [HNR68] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [HZ04] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, UK, 2 edition, 2004.
- [Int] Intel Corporation. Intel core ultra 9 processor 275hx – product specifications. <https://www.intel.com/content/www/us/en/products/sku/242293/intel-core-ultra-9-processor-275hx-36m-cache-up-to-5-40-ghz/specifications.html>. Accesat: mai 2025.
- [Int23] International Guide Dog Federation. Annual statistics report on global guide dog mobility. Technical report, IGDF, 2023.

-
- [LCL⁺25] Haotong Lin, Sili Chen, Junhao Liew, Donny Y Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.
- [Met] Meta Platforms, Inc. Ray-ban meta smart glasses. <https://www.meta.com/smart-glasses/>. Accesat: mai 2025.
- [ODM⁺24] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [Rasa] Raspberry Pi Ltd. Raspberry pi 5 – product brief. <https://www.raspberrypi.com/products/raspberry-pi-5/>. Accesat: mai 2025.
- [Rasb] Raspberry Pi Ltd. Raspberry pi ai hat+ – product brief. <https://www.raspberrypi.com/products/ai-hat/>. Accesat: mai 2025.
- [S⁺18] G. Stevens et al. Barriers to accessing assistive technology for people with visual impairment. *Disability and Rehabilitation*, 40, 2018.
- [S⁺21] J. D. Steinmetz et al. Causes of blindness and vision impairment in 2020 and trends over 30 years. *Scientific Reports (Nature)*, 2021.
- [SHKF12] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor segmentation and support inference from rgbd images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 746–760, 2012.
- [Wor19] World Health Organization. *World report on vision*. WHO Press, Geneva, 2019.
- [YKH⁺24a] Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10371–10381, 2024.
- [YKH⁺24b] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems*, 37:21875–21911, 2024.
-

- [Zha00] Zhengyou Zhang. A flexible new technique for camera calibration. *IEEE Transactions on pattern analysis and machine intelligence*, 22(11):1330–1334, 2000.